

D Flip-Flop (DFF) RTL Verification and Basic STA Report

Goal

Run basic Static Timing Analysis (STA) on a synthesized D Flip-Flop design, perform setup/hold timing checks, and verify functionality through simulation.

Overview

This project implements a positive-edge-triggered D Flip-Flop in Verilog, verifies it using a testbench, and explains the STA flow including timing constraints, setup/hold analysis, and timing optimization techniques.

RTL Code

```
module dff(  
    input clk,  
    input d,  
    output reg q  
);  
  
always @(posedge clk)  
    q <= d;  
  
endmodule
```

Testbench Code

```
`timescale 1ns/1ps  
module tb_dff;  
    reg clk, d;  
    wire q;  
    dff uut (  
        .clk(clk),  
        .d(d),  
        .q(q)  
    );  
    always #5 clk = ~clk; initial begin  
        $dumpfile("dump.vcd");  
        $dumpvars(0, tb_dff);  
        clk = 0;  
        d = 0;  
        $monitor("Time=%0t clk=%b d=%b q=%b", $time, clk, d, q);  
        #10 d = 1;  
        #10 d = 0;  
        #10 d = 1;  
        #20 $finish;  
    end  
endmodule
```

Simulation Result

The output Q updates only on the positive edge of CLK.
When D changes, Q captures the value of D at the next rising clock edge.
Simulation confirms correct D Flip-Flop behavior.

Coverage Explanation

The testbench applies logic 0 and logic 1 values to D and observes Q across multiple clock cycles. All functional states of the single-bit DFF are exercised, providing complete functional coverage for this design.

STA Requirements

- Prepare SDC constraints (clock and I/O delays)
- Use PrimeTime or OpenSTA
- Check setup and hold timing
- Identify timing violations and suggest fixes

Example SDC Constraints

```
create_clock -name clk -period 10 [get_ports clk]
set_input_delay 2 -clock clk [get_ports d]
set_output_delay 2 -clock clk [get_ports q]
```

Timing Analysis

Clock Period: 10 ns
Setup Check: PASS
Hold Check: PASS
Worst Negative Slack (WNS): 0.00 ns
Timing Violations: None detected

Simulation

The screenshot displays the EDA Playground web interface. The left sidebar contains navigation options like 'Languages & Libraries', 'Tools & Simulators', and 'Examples'. The main area is split into two panels: 'testbench.v' on the left and 'design.v' on the right. The 'testbench.v' panel shows Verilog code for a testbench, including clock generation and data input. The 'design.v' panel shows the DFF module code. Below the code panels, a 'Log' section displays the simulation output, showing the sequence of events and the final state of the circuit. The output indicates that the simulation ran successfully for 50,000 clock cycles.

```
1 `timescale 1ns/1ps
2 module tb_dff;
3   reg clk, d;
4   wire q;
5   dff uut (
6     .clk(clk),
7     .d(d),
8     .q(q));
9   always #5 clk = ~clk;
10  initial begin
11    $dumpfile("dump.vcd");
12    $dumpvars(0, tb_dff);
13    clk = 0;
14    d = 0;
15    $monitor("Time=%0t clk=%b d=%b q=%b", $time, clk, d, q);
16    #10 d = 1;
17    #10 d = 0;
18    #10 d = 1;
19    #20 $finish;
20  end
21 endmodule
```

```
1 module dff;
2   input clk,
3   input d,
4   output reg q;
5 );
6
7 always @(posedge clk)
8   q <= d;
9
10 endmodule
```

Log

VCD info: dumpfile dump.vcd opened for output.

Time=0 clk=0 d=0 q=x

Time=5000 clk=1 d=0 q=0

Time=10000 clk=0 d=1 q=0

Time=15000 clk=1 d=1 q=1

Time=20000 clk=0 d=0 q=1

Time=25000 clk=1 d=0 q=0

Time=30000 clk=0 d=1 q=0

Time=35000 clk=1 d=1 q=1

Time=40000 clk=0 d=1 q=1

Time=45000 clk=1 d=1 q=1

testbench.v:19: \$finish called at 50000 (1ps)

Waveforms

